

Аудит безопасности веб-приложения "Анкета программиста"

Введение

В данном отчете представлены результаты аудита безопасности веб-приложения, а также исправления выявленных уязвимостей. Приложение разработано на PHP с использованием MySQL и включает функционал регистрации пользователей, авторизации и админ-панели.

1. Защита от XSS (Cross-Site Scripting)

Описание уязвимости

XSS (межсайтовый скриптинг) — это тип уязвимости, при котором злоумышленник может внедрить вредоносный код на веб-страницу, которая затем выполняется в браузере других пользователей. Основная причина возникновения XSS — недостаточная обработка пользовательского ввода перед его выводом на страницу.

Уязвимость 1.1: Вывод данных из cookie

Файл: index.php

```
14     if (!empty($_COOKIE['save'])) {
15         $messages[] = 'Результаты сохранены.';
16         if (!empty($_COOKIE['login']) && !empty($_COOKIE['pass'])) {
17             $messages[] = sprintf(
18                 'Можно <a href="login.php">войти</a> с логином <strong>%s</strong> и паролем <strong>%s</strong>',
19                 strip_tags($_COOKIE['login']),
20                 strip_tags($_COOKIE['pass'])
21             );
22         }
```

Почему уязвимо:

Функция `strip_tags()` удаляет только полные HTML-теги, но не экранирует специальные символы. Злоумышленник может подставить значение с атрибутами тегов (например, `img src=x onerror=...`) или использовать HTML-сущности для обхода. Cookie могут быть подменены через XSS в другом месте или другими методами.

Исправление:

```
if (!empty($_COOKIE['save'])) {
    $messages[] = 'Результаты сохранены.';
    if (!empty($_COOKIE['login']) && !empty($_COOKIE['pass'])) {
        $messages[] = sprintf(
            'Можно <a href="login.php">войти</a> с логином <strong>%s</strong> и паролем <strong>%s</strong>',
            htmlspecialchars($_COOKIE['login'], ENT_QUOTES, 'UTF-8'),
            htmlspecialchars($_COOKIE['pass'], ENT_QUOTES, 'UTF-8')
        );
    }
}
```

2. Information Disclosure

Information Disclosure (раскрытие информации) — это уязвимость, при которой система выдает злоумышленнику техническую информацию о своей внутренней структуре, которая может быть использована для организации

более сложных атак. Проблема возникает, когда детальные сообщения об ошибках попадают к конечному пользователю.

Уязвимость 2.1: Ошибка подключения к БД

Файл: config.php

```
15         try {
16             $db = new PDO('mysql:host=' . DB_HOST . ';dbname=' . DB_NAME, DB_USER, DB_PASS, [
17                 PDO::ATTR_PERSISTENT => true,
18                 PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
19             ]);
20         } catch (PDOException $e) {
21             die('Ошибка подключения к базе данных: ' . $e->getMessage());
22         }
```

Почему уязвимо:

Метод `$e->getMessage()` возвращает полный текст ошибки MySQL, включая имя пользователя БД, тип ошибки, иногда путь к файлу.

Исправление:

```
15         ini_set('display_errors', 0);
16         ini_set('display_startup_errors', 0);
17         error_reporting(E_ALL);
18
19         ini_set('log_errors', 1);
20         ini_set('error_log', __DIR__ . '/logs/php_errors.log');
21
22         if (!is_dir(__DIR__ . '/logs')) {
23             mkdir(__DIR__ . '/logs', 0755);
24         }
25
26         try {
27             $db = new PDO('mysql:host=' . DB_HOST . ';dbname=' . DB_NAME, DB_USER, DB_PASS, [
28                 PDO::ATTR_PERSISTENT => true,
29                 PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
30             ]);
31         } catch (PDOException $e) {
32             error_log('Database connection error: ' . $e->getMessage());
33             die('Технические работы. Пожалуйста, попробуйте позже.');
```

Уязвимость 2.2: Ошибка при удалении пользователя

Файл: admin_functions.php

```
53         } catch (PDOException $e) {
54             $db->rollBack();
55             return ['success' => false, 'message' => 'Ошибка удаления: ' . $e->getMessage()];
56         }
```

Почему уязвимо:

Сообщение об ошибке возвращается в `admin.php` и выводится на экран администратора, раскрывая детали SQL-запроса и структуру БД.

Исправление:

```
53     } catch (PDOException $e) {
54         $db->rollBack();
55         error_log('Delete user error (ID: ' . $id . '): ' . $e->getMessage());|
56         return ['success' => false, 'message' => 'Ошибка при удалении пользователя. Подробности в логе.'];
57     }
```

Уязвимость 2.3: Ошибка при обновлении пользователя

Файл: admin_functions.php

```
85     } catch (PDOException $e) {
86         return ['success' => false, 'message' => 'Ошибка обновления: ' . $e->getMessage()];
87     }
```

Почему уязвимо:

Аналогично пункту 2.2.

Исправление:

```
86     } catch (PDOException $e) {
87         error_log('Update user error (ID: ' . $id . '): ' . $e->getMessage());
88         return ['success' => false, 'message' => 'Ошибка при обновлении данных пользователя. Подробности в логе.'];
89     }
```

Уязвимость 2.4: Ошибка в index.php

Файл: index.php

```
247     } catch (PDOException $e) {
248         $db->rollBack();
249         die('Ошибка базы данных: ' . $e->getMessage());
250     } catch (Exception $e) {
251         $db->rollBack();
252         die('Ошибка: ' . $e->getMessage());
253     }
```

Почему уязвимо:

Полный текст исключения выводится пользователю. При SQL-инъекции злоумышленник увидит синтаксические ошибки, что поможет ему отладить атаку.

Исправление:

```
247     } catch (PDOException $e) {
248         $db->rollBack();|
249         error_log('Database error in form processing: ' . $e->getMessage());
250         die('Произошла ошибка при сохранении данных. Пожалуйста, попробуйте позже.');
```

```
251     } catch (Exception $e) {
252         $db->rollBack();
253         error_log('General error in form processing: ' . $e->getMessage());
254         die('Произошла непредвиденная ошибка. Администратор уже уведомлён.');
```

```
255     }
```

3. SQL Injection

SQL-инъекция — это тип атаки, при котором злоумышленник внедряет произвольный SQL-код в запрос к базе данных. Проблема возникает, когда пользовательские данные напрямую подставляются в SQL-запрос без должной обработки, что позволяет изменить логику выполнения запроса.

Уязвимость 3.1: Использование MD5 для паролей администратора

Файл: auth.php

```
18      $stmt = $db->prepare("SELECT password_hash FROM admins WHERE username = ?");
19      $stmt->execute([$ _SERVER['PHP_AUTH_USER']]);
20      $admin = $stmt->fetch(PDO::FETCH_ASSOC);
21
22      if (!$admin || md5($_SERVER['PHP_AUTH_PW']) !== $admin['password_hash']) {
```

Почему уязвимо:

MD5 — устаревший алгоритм хеширования. Он слишком быстрый (легко перебирать пароли), подвержен коллизиям, не использует соль. При утечке базы данных пароли восстанавливаются за минуты.

Исправление:

```
18      $stmt = $db->prepare("SELECT password_hash FROM admins WHERE username = ?");
19      $stmt->execute([$ _SERVER['PHP_AUTH_USER']]);
20      $admin = $stmt->fetch(PDO::FETCH_ASSOC);
21
22      if (!$admin || !password_verify($_SERVER['PHP_AUTH_PW'], $admin['password_hash'])) {
```

Уязвимость 3.2: Использование MD5 для генерации паролей пользователей

Файл: index.php

```
213          } else {
214              $login = 'user_' . rand(1000, 9999) . uniqid();
215              $pass = substr(md5(uniqid(rand(), true)), 0, 8);
216              $pass_hash = md5($pass);
217
```

Почему уязвимо:

Та же проблема — MD5. Кроме того, `uniqid()` не является криптостойким, её можно предсказать. Пароль длиной 8 символов из MD5-хеша имеет недостаточную энтропию.

Исправление:

```

213     } else {
214         $login = 'user_' . bin2hex(random_bytes(8));
215
216         $characters = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!@#%&*()';
217         $pass = '';
218         for ($i = 0; $i < 12; $i++) {
219             $pass .= $characters[random_int(0, strlen($characters) - 1)];
220         }
221
222         $pass_hash = password_hash($pass, PASSWORD_DEFAULT);

```

Уязвимость 3.3: Отсутствие валидации ID

Файл: admin_functions.php

```

24  ✓ function getUserById($id) {
25      $db = getDB();
26      $stmt = $db->prepare("SELECT * FROM users WHERE id = ?");
27      $stmt->execute([$id]);
28      $user = $stmt->fetch(PDO::FETCH_ASSOC);

```

Почему уязвимо (частично):

Подготовленные выражения используются — это хорошо. Но нет проверки, что \$id действительно число. Если передать строку, PDO преобразует её, но это может привести к неожиданному поведению.

Уязвимость 3.4: Прямой query() без параметров

Файл: admin_functions.php

```

18  function getAllUsers() {
19      $db = getDB();
20      return $db->query("SELECT id, name, phone, email, birthdate, sex, biography, login, created_at FROM users ORDER BY id DESC")->fetchAll(PDO::FETCH_ASSOC);
21  }

```

Почему уязвимо (потенциально):

В этом конкретном запросе нет пользовательского ввода, поэтому он безопасен. Но использование query() вместо prepare() — опасная практика. Если в будущем в этот запрос добавится параметр через конкатенацию, возникнет уязвимость.

4. CSRF (Cross-Site Request Forgery) - 2 балла

CSRF (межсайтовая подделка запроса) — это атака, при которой злоумышленник заставляет браузер аутентифицированного пользователя отправить HTTP-запрос на целевой сайт без ведома пользователя. Проблема возникает, когда сервер не проверяет, что запрос исходит именно с оригинального сайта, а не с подконтрольного злоумышленнику ресурса.

Уязвимость 4.1: Удаление пользователя через GET

Файл: admin.php

```

15     if ($action === 'delete' && $user_id) {
16         $result = deleteUser($user_id);
17         if ($result['success']) {
18             $message = $result['message'];
19         } else {
20             $error = $result['message'];
21         }
22     }

```

Почему уязвимо:

Изменение данных (удаление пользователя) выполняется через GET-запрос. Злоумышленник может разместить на своём сайте тег ``, и браузер администратора автоматически отправит этот запрос.

Уязвимость 4.2: Редактирование пользователя без CSRF-токена

Файл: admin.php

```

24     if ($action === 'edit' && $_SERVER['REQUEST_METHOD'] === 'POST') {
25         $result = updateUser($user_id, $_POST);
26         if ($result['success']) {
27             $message = $result['message'];
28             $action = '';
29         } else {
30             $error = $result['message'];
31         }
32     }

```

Почему уязвимо:

POST-запрос на редактирование не содержит CSRF-токен. Злоумышленник может создать на своём сайте форму, которая отправляется на admin.php, и заставить администратора её отправить.

Уязвимость 4.3: Форма регистрации без CSRF-токена

Файл: form.php

```

45
46     <form action="" method="POST">

```

Файл: index.php

```

83     else {
84         $errors = FALSE;
85
86         if (empty($_POST['name'])) {
87             setcookie('name_error', '1', time() + 24 * 60 * 60);
88             $errors = TRUE;
89         } elseif (strlen($_POST['name']) > 150) {
90             setcookie('name_error', '1', time() + 24 * 60 * 60);
91             $errors = TRUE;
92         } elseif (!preg_match('/^[a-zA-Za-яА-ЯёЁ\s]+$/', $_POST['name'])) {
93             setcookie('name_error', '1', time() + 24 * 60 * 60);
94             $errors = TRUE;
95         } else {
96             setcookie('name_value', $_POST['name'], time() + 30 * 24 * 60 * 60);
97         }
98
99         if (!empty($_POST['phone']) && !preg_match('/^[+0-9\s\-\(\)]{10,20}$/', $_POST['phone'])) {
100             setcookie('phone_error', '1', time() + 24 * 60 * 60);
101             $errors = TRUE;
102         } else {
103             setcookie('phone_value', $_POST['phone'], time() + 30 * 24 * 60 * 60);
104         }
105
106         if (!empty($_POST['email']) && !filter_var($_POST['email'], FILTER_VALIDATE_EMAIL)) {
107             setcookie('email_error', '1', time() + 24 * 60 * 60);
108             $errors = TRUE;
109         } else {
110             setcookie('email_value', $_POST['email'], time() + 30 * 24 * 60 * 60);
111         }
112
113         if (!empty($_POST['birthdate'])) {
114             $date = DateTime::createFromFormat('Y-m-d', $_POST['birthdate']);
115             if (!$date || $date->format('Y-m-d') !== $_POST['birthdate']) {
116                 setcookie('birthdate_error', '1', time() + 24 * 60 * 60);

```

Почему уязвимо:

Форма отправки анкеты не содержит CSRF-токен. Злоумышленник может создать поддельную форму на своём сайте и заставить аутентифицированного пользователя отправить данные.

Уязвимость 4.4: Форма входа без CSRF-токена

Файл: login.php

```

41         <form action="" method="post">
42             <div class="form-group">
43                 <label for="login">Логин</label>
44                 <input name="login" id="login" type="text" required />
45             </div>
46
47             <div class="form-group">
48                 <label for="pass">Пароль</label>
49                 <input name="pass" id="pass" type="password" required />
50             </div>
51
52             <button type="submit" class="login-btn">Войти</button>
53         </form>

```

Почему уязвимо:

Хотя CSRF для формы входа менее критичен (злоумышленник не может украсть пароль, но может заставить жертву войти в свой аккаунт), это всё равно проблема безопасности.

5. Include и Upload уязвимости - 1 балл

Include уязвимости (LFI/RFI) — позволяют злоумышленнику включать локальные или удаленные файлы через динамическое подключение на основе пользовательского ввода. Upload уязвимости — возникают при загрузке файлов на сервер без достаточных проверок их типа, размера и содержимого.

Уязвимости Include (LFI/RFI)

Поиск в коде конструкций: require, require_once, include, include_once

Результат анализа:

Во всех файлах используется только жестко заданные имена файлов. Нет ни одного места, где имя файла для подключения берётся из \$_GET, \$_POST, \$_COOKIE.

Вывод: УЯЗВИМОСТЕЙ INCLUDE НЕТ

Уязвимости Upload

Поиск в коде конструкций: \$_FILES, move_uploaded_file(), <input type="file">

Результат анализа:

\$_FILES — не используется ни в одном файле

move_uploaded_file() — не используется

<input type="file"> — отсутствует в HTML

Вывод: УЯЗВИМОСТЕЙ UPLOAD НЕТ

